REFERENCIA DEL SISTEMA

Cómo funciona Dupla OS

Arquitectura, entornos, flujos de usuario y automatizaciones. Todo lo que necesitas saber para usar el sistema con profundidad.

← Guía de inicio rápido

Mapa del sistema

CONCEPTO

El sistema en una idea

Dupla OS es un **arnés de markdown**. El workflow vive en archivos `.md` — no en el LLM ni en el IDE. Conectas cualquier IA como "cerebro" y el sistema funciona igual: contexto, estado, memoria, skills.

Archivos .md como estado

`PROJECT_STATE.md` es la memoria. `ROADMAP.md` es el plan. El LLM los lee al inicio de cada sesión — sin que tengas que re-explicar nada.

Skills como programas de prompt

Cada skill es un archivo `.md` con instrucciones. El LLM lo ejecuta cuando escribes `/comando`. Cero código, cero dependencias.

Hooks como automatizaciones

Scripts bash que corren *fuera del contexto* — sin quemar tokens. Avisan cuando hay que hacer checkpoint, guardan snapshots, sincronizan entre modelos.

```
# Estructura de un proyecto Dupla OS
mi-proyecto/
```

```
├── CLAUDE.md                # Contexto del proyecto (lo que el LLM sabe siempre)
├── claude-progress.txt      # Estado ligero — leído en ~30 tokens
├── docs/
│   ├── PROJECT_STATE.md     # Fuente de verdad: dónde estás, qué sigue, qué bloquea
│   ├── ROADMAP.md           # Fases + GO/NO-GO checkpoints (dinámico)
│   ├── ARCHITECTURE.md      # Decisiones técnicas (solo si es software)
│   ├── SPEC.md              # Problema · Usuario · MVP · Kill condition
│   └── PROBLEMS.md          # Bugs y soluciones reusables
└── .agents/rules/claude.md  # Copia para Gemini/Antigravity
```

Por qué markdown y no una base de datos o app

Los archivos .md son universales — cualquier LLM los lee. Git los versiona. Funcionan sin internet. No dependen de que una empresa mantenga una plataforma. El sistema sobrevive a que cambies de Claude a Gemini a GPT-4 sin perder nada.

SKILLS

Cómo funciona un skill

Un skill es un archivo `nombre-skill.md` con instrucciones precisas para el LLM. Cuando escribes `/nuevo-proyecto`, el LLM lee ese archivo y lo ejecuta como un programa.

Escribes /new-session → LLM carga ~/.claude/skills/new-session.md → Lee PROJECT_STATE + ROADMAP fase → Genera plan del día (~12 líneas)

Cuándo se cargan

Solo cuando los invocas. **Costo cero hasta que los usas.** Un skill de 200 líneas cuesta ~500 tokens — solo en el momento del invocación. El resto del tiempo no existen en tu contexto.

Dónde viven

Claude Code: `~/.claude/skills/*.md`
Antigravity: `~/.gemini/antigravity/global_workflows/`
Cursor: `.cursor/commands/*.md`
Todos instalados por `bin/install.sh`.

SKILL	QUÉ HACE EXACTAMENTE	CUÁNDO
/new-session	Lee SESSION block (~60t) + fase actual ROADMAP (~100t). Detecta commits sin checkpoint → reconstruye desde git. Genera plan en ≤12 líneas.	Cada sesión
/checkpoint	Actualiza PROJECT_STATE desde git log. Sincroniza avance ROADMAP. Detecta GO/NO-GO si fase completada. Actualiza ARCHITECTURE/PROBLEMS solo si hay señales. Propone commit.	Cada sesión
/new-project	Assessment IML (7 preguntas, 6 dimensiones, score Verde/Amarillo/Rojo). Genera SPEC.md, ROADMAP con GO/NO-GO, ARCHITECTURE, PROJECT_STATE. Stack adaptado a constraints reales.	Por proyecto
/setup-dupla	Entrevista en bloques (identidad, stack, proyectos, plugins). Genera CLAUDE.md + SYSTEM.md + MEMORY_GLOBAL.md. Detecta IDE y llama a install.sh automáticamente.	Una vez
/adapt-project	Detecta QUICKSTATE.md (micro → full). Lee estructura actual, mapea docs viejos, genera lo que falta. Muestra diff antes de cada cambio. Nunca borra — archiva.	Onboarding
/quick-start	Crea QUICKSTATE.md con 2 preguntas. Sin docs/, sin git obligatorio. Si >14 días activo, sugiere escalar a /adapt-project. Micro → full sin perder nada.	Micro sesiones
/health-check	6 checks: herramientas, global, proyecto, equipo, coherencia stack/código, evaluador (SESSION vs git, hooks en disco, archivos huérfanos). CRITICAL / WARNING / INFO.	Mantenimiento
/update-dupla	Compara versión local vs GitHub. Backup automático. Re-detecta IDEs. Actualiza skills + hooks en todos los entornos. Propaga templates cambiados a proyectos (aditivo, nunca sobreescribe).	Por versión
/knowledge-graph	Usa graphify para transformar docs/ en graph.html (visual), graph.json (legible por AI) y GRAPH_REPORT.md. 5× más eficiente en tokens que leer todos los docs por separado.	Por fase
/restore	Lista save points (git stash o _versions/ si sin git). Permite revertir estado del proyecto a un punto anterior.	Si algo falla

SKILL	QUÉ HACE EXACTAMENTE	CUÁNDO
`/token-budget`	Avisa cuando el contexto se acerca al límite y propone acciones más livianas. Activo en Claude Code.	Automático

HOOKS

Automatizaciones fuera del contexto

Los hooks son scripts bash que corren automáticamente en eventos del IDE — **sin usar tokens**. Solo disponibles en Claude Code (IDE/CLI). En otros entornos, hay que hacerlos manualmente.

suggest-checkpoint.sh

Evento: `Stop` (cuando Claude termina de responder)

Cuenta commits desde el último checkpoint. Si hay >5 commits sin guardar → avisa. Si hay cualquier commit → avisa cada 3 stops. Usa contador en `/tmp/` para no spamear.

auto-snapshot.sh

Evento: `Stop`

Copia `docs/` a `_versions/YYYY-MM-DD_HH-MM/` en cada stop. Sin tocar git. Funciona incluso en proyectos sin git. Es el historial de seguridad de tus docs.

guard-project-state.sh

Evento: `PreToolUse (Write)`

Antes de que Claude escriba cualquier archivo: verifica si el proyecto tiene `CLAUDE.md`. Si no existe → bloquea el write. Evita que el LLM cree archivos en proyectos no inicializados.

sync-gemini.sh

Evento: `PostToolUse (Write)`

Después de que Claude escribe `~/.claude/CLAUDE.md`: copia automáticamente a `~/.gemini/GEMINI.md`. Tu identidad global siempre sincronizada entre Claude y Gemini.

```
# Hooks configurados en ~/.claude/settings.json
{
  "hooks": {
    "Stop":            ["suggest-checkpoint.sh", "auto-snapshot.sh"],
    "PreToolUse":      { "Write": "guard-project-state.sh" },
    "PostToolUse":     { "Write": "sync-gemini.sh" },
    "UserPromptSubmit": ["session-reminder.sh"]
  }
```

```
}
# Todos cross-platform: python3 primary, cksum/date fallback (Windows compatible)
```

En Cursor, Windsurf y Chat: compensación manual

No hay hooks automáticos fuera de Claude Code. La compensación: ejecutar `/checkpoint` manualmente al cerrar cada sesión. El sistema no te pierde el estado — pero no te avisa automáticamente.

ENTORNOS

Qué puedes hacer en cada entorno

El sistema funciona en todos los entornos. Lo que cambia es el nivel de automatización disponible. El estado (docs, git) es universal — no depende del entorno.

NIVEL DE AUTOMATIZACIÓN

Claude Code IDE/CLI — Skills + Hooks + MCP → **Antigravity / Gemini** — Skills + Shell → **Cursor / Windsurf** — Skills (sin hooks) → **Chat / Web** — Solo texto + adjuntos

ENTORNO	SKILLS	HOOKS	LEER ARCHIVOS	AGENTES PARALELOS	HANDOFF
Claude Code IDE/CLI Nivel: Completo	✅ /comando nativo	✅ 4 hooks automáticos	✅ directo + MCP	✅ Agent tool nativa	✅
Antigravity / Gemini Nivel: Alto	✅ workflows	❌ manual	✅ run_command	⚠️ run_command	✅
Cursor Nivel: Medio	✅ Agent Mode	❌ manual	✅ Agent Mode	⚠️ múltiples tabs	✅
Windsurf Nivel: Medio	⚠️ @workflow	❌ manual	✅ Cascade	❌	✅

ENTORNO	SKILLS	HOOKS	LEER ARCHIVOS	AGENTES PARALELOS	HANDOFF
GPT-4 / API Nivel: Base	❌ pegar texto	❌	⚠️ adjuntar	❌	✅
Web Chat / Mobile Nivel: Mínimo	❌	❌	⚠️ adjuntar	❌	✅

Claude Code — cómo instalar

```
git clone https://github.com/leo1z/dupla-workflow
bash dupla-workflow/bin/install.sh
```

Detecta VS Code, Antigravity y Cursor automáticamente. Instala skills + hooks en una pasada.

Cursor — clave: usa Agent Mode

Skills disponibles en `.cursor/commands/`. **Chat Mode y Composer no leen rules** — siempre usa Agent Mode para que los skills funcionen.

⚠️ Chat Mode = no funciona

✅ Agent Mode = OK

Antigravity — skills como texto

Si el skill no se activa con /nombre, simplemente escribe el nombre del skill como texto — Gemini lo lee como instrucción y lo ejecuta igual. GEMINI.md se carga automáticamente.

Web Chat — solo continuidad

Adjunta `PROJECT_STATE.md` + pega el `<handoff>` block. Útil para consultas rápidas o cuando estás en movimiento. No intentes construir desde aquí.

FLUJOS

Flujos de usuario comunes

Cada flujo está cubierto por el sistema. Aquí cómo funciona cada uno paso a paso.

Sesión de trabajo diaria

Uso diario

1 Abres el IDE y escribes

`/new-session`

Lee SESSION block de PROJECT_STATE + fase actual del ROADMAP. Si hay commits sin checkpoint, reconstruye el estado desde git. Genera plan del día en ≤12 líneas.

2 **Trabajas**

Sigues el plan. El hook `session-reminder.sh` avisa si el estado tiene >24h. `guard-project-state.sh` bloquea writes si el proyecto no está inicializado.

3 **Al cerrar, guardas**

`/checkpoint`

Modo 1 (Quick) o Modo 2 (Full Close). Actualiza PROJECT_STATE, sincroniza ROADMAP, propone mensaje de commit. Si completaste una fase → avisa GO/NO-GO.

4 **git push → próxima sesión retoma aquí**

Crear un proyecto nuevo

Por proyecto

Idea nueva

IML Assessment (7 preguntas) → score Verde/Amarillo/Rojo → si verde: entrevista completa → stack adaptado → docs generados

Ya investigada

Salta Phase 0.5. Pide dónde está el research. Genera SPEC + ROADMAP + stack desde lo que ya tienes.

MVP definido

Salta Phases 2+4. Pide ROADMAP y ARCHITECTURE existentes. Solo genera lo que falta.

El ROADMAP siempre tiene GO/NO-GO

Cada fase en el ROADMAP tiene: Outcomes (qué entregar), Continue if (criterios para seguir), Kill if (cuándo parar). /checkpoint lo actualiza automáticamente cuando detecta que entregaste un Outcome en git.

De proyecto pequeño a proyecto completo

Escalado

/quick-start QUICKSTATE.md → 14+ días activo o proyecto crece → /adapt-project detecta QUICKSTATE → docs/ completo QUICKSTATE archivado

`/adapt-project` detecta `QUICKSTATE.md` automáticamente y usa su contenido (What, Done, Next) para pre-llenar PROJECT_STATE. El archivo original se archiva en `docs/ARCHIVE.md` — nada se pierde.

Actualizar el sistema y propagar a proyectos

Mantenimiento

1 Verificar versión (sin actualizar)

```
/update-dupla check
```

2 Actualizar con backup automático

```
/update-dupla
```

Re-detecta IDEs. Actualiza skills en Claude Code + Antigravity + Cursor. Backup en `~/.claude/skills-backup/v[old]/` antes de tocar nada.

3 Propagación automática a proyectos

Si algún template cambió en la nueva versión, el sistema detecta qué proyectos usan ese template y propone agregar los nuevos campos — **sin sobreescribir contenido existente**. Siempre aditivo. Siempre con aprobación.

Se olvidó el checkpoint

Cubierto automáticamente

3 capas de protección, sin intervención del usuario:

1. **suggest-checkpoint.sh** — avisa cada 3 stops si hay commits nuevos sin guardar
2. **session-reminder.sh** — avisa al inicio de la siguiente sesión si el estado tiene >24h
3. **/new-session** — reconstruye SESSION.Done automáticamente desde git log si hay commits más nuevos que el estado guardado

Cambiar de LLM o entorno sin perder contexto

El `<handoff>` block es el puente entre entornos. Funciona en absolutamente todos — incluso en WhatsApp o Telegram.

Cómo generar un handoff

1. En el entorno actual: `/checkpoint` → opción Handoff
2. Copia el bloque `<handoff>` generado
3. En el nuevo entorno: pega el bloque + escribe `/new-session`

```
# Estructura del bloque handoff
<handoff>
Date:    2026-05-03
From:    Claude (sesión 8)
To:      Next session
Project: c:\Users\Leo\Projects\mi-app
Goal:    Implementar auth con Supabase
Branch:  work
Done:    Login UI completo · API endpoint
Next:    Conectar frontend con endpoint /auth
Context: Leer docs/PROJECT_STATE.md primero
</handoff>
```

DESTINO	CÓMO RECIBIR EL HANDOFF
Claude Code IDE/CLI	Nuevo chat → navega al proyecto → pega <handoff> → escribe `/new-session`
Antigravity / Gemini	Nuevo chat → pega <handoff> → escribe "Ejecuta new-session" (o el nombre del skill)
Cursor	Nuevo chat en **Agent Mode** → pega <handoff> → escribe `/new-session`
Web Chat / GPT	Adjunta PROJECT_STATE.md → pega <handoff> + "Continúa desde Next:"
WhatsApp / Telegram	Pega <handoff> block + contenido de claude-progress.txt → "Continúa desde Next:"

sync-gemini: handoff automático para CLAUDE.md

Cuando Claude Code escribe `~/.claude/CLAUDE.md`, el hook `sync-gemini.sh` copia automáticamente a `~/.gemini/GEMINI.md`. Tu identidad siempre sincronizada — sin hacer nada manualmente.

MULTI-AGENTE

Trabajo paralelo y patrón de roles

Dupla OS no impone un framework de agentes — usa las capacidades nativas de cada entorno. El patrón recomendado es siempre el mismo.

Planificador

Lee PROJECT_STATE + ROADMAP → divide trabajo en tareas independientes con contexto completo

→

Implementadores

Cada uno recibe tarea + rama + archivos a tocar. Trabajan aislados.

→

Evaluador

/health-check verifica coherencia entre lo hecho y el estado declarado antes del merge.

Claude Code — paralelismo nativo

Agent tool con `run_in_background: true`. Cada subagente recibe contexto completo en su prompt. Los resultados se consolidan al final.

Cursor / Antigravity — paralelismo manual

Abre múltiples chats (tabs) en Agent Mode. Cada tab trabaja en su rama. Consolidas al final con /checkpoint antes del merge.

Regla clave del paralelismo

Cada agente paralelo necesita su propio contexto completo. No asumas que un subagente sabe lo que hizo el otro. Siempre /checkpoint antes del merge para verificar coherencia.

ROADMAP

Roadmap dinámico con checkpoints

El ROADMAP no es un documento estático — se actualiza solo en cada /checkpoint cuando el LLM detecta Outcomes completados en el git log.

```
# Estructura de cada fase en ROADMAP.md
## Phase 1 — MVP

Outcomes:
[x] Login funcional con Supabase        ← marcado por /checkpoint
[ ] Dashboard con datos reales
```

```
GO/NO-GO Checkpoint:
  ✅ Continue if: 3+ usuarios activados en la semana
  ⚠️ Adapt if:  usuarios activados pero no retornan → revisar onboarding
  ❌ Kill if:    la asunción "las empresas pagarían $X" resulta falsa
```

Auto-actualización

/checkpoint compara git log con Outcomes de la fase. Si un Outcome aparece en los commits → lo marca `[x]` automáticamente.

GO/NO-GO alerta

Si todos los "Continue if" están cumplidos → Phase_Status pasa a `GO/NO-GO Pending` y /new-session lo muestra como alerta prioritaria.

Kill conditions

Cada fase tiene una condición que mata el proyecto. Las respuestas del IML Assessment alimentan directamente el Kill if de Phase 1.

IML ASSESSMENT

Evaluación de madurez de idea

Antes de construir cualquier cosa, /new-project evalúa si la idea vale el esfuerzo. 7 preguntas, 6 dimensiones, 3 salidas posibles.

Verde — construye

4+ dimensiones fuertes. El sistema lista las fortalezas y el riesgo principal a monitorear. Continúa a entrevista completa.

Amarillo — con advertencia

2-3 dimensiones fuertes. El sistema detalla qué gaps hay y ofrece: continuar asumiendo riesgos, validar primero, o salir.

Rojo — valida primero

0-1 dimensiones fuertes. Stop. El sistema da acciones concretas de validación y un prompt para ejecutarlas ahora mismo.

Las 6 dimensiones evaluadas

Problema
¿Específico con momento concreto, o vago y genérico?

Validación
¿3+ personas reales confirmaron, o solo opinión propia?

Disposición a pagar

¿Cifra concreta, o "tal vez / depende"?

Mercado

¿Alternativas claras con gaps, o "no existe nada igual"?

Ventaja real

¿Acceso exclusivo o skill específico, o "soy muy apasionado"?

Riesgo nombrado

¿Asunción específica que si es falsa mata el proyecto?

Guía de inicio rápido → Mapa del sistema →